

ĐỒ ÁN TỐT NGHIỆP

SINH MÔ HÌNH 3D TỪ MÔ TẢ ĐA PHƯƠNG THỨC

GVHD: TS. LÊ THÀNH SÁCH

GVPB: TS. NGUYỄN ĐỨC DŨNG

Sinh viên: Mai Phương Nhã, Huỳnh Nguyên Phúc

Khoa Khoa học và Kỹ thuật Máy tính
Trường Đại học Bách khoa
Đại học Quốc gia Thành phố Hồ Chí Minh

Tháng 5 – Năm 2025

Table of Contents

- 1 Đặt vấn đề
- 2 Các hướng giải quyết
 - Sinh vật thể 3D: Phương pháp tối ưu hóa
 - Sinh vật thể 3D: Phương pháp sinh trực tiếp
- 3 Phương pháp
- 4 Thực nghiệm
 - Tập dữ liệu và tiền xử lý
 - Các thí nghiệm
- 5 Đánh giá
- 6 Kết luận

Table of Contents

1 Đặt vấn đề

2 Các hướng giải quyết

- Sinh vật thể 3D: Phương pháp tối ưu hóa
- Sinh vật thể 3D: Phương pháp sinh trực tiếp

3 Phương pháp

4 Thực nghiệm

- Tập dữ liệu và tiền xử lý
- Các thí nghiệm

5 Đánh giá

6 Kết luận

Tầm quan trọng

🏢 Vì sao 3D lại thiết yếu?

🏠 Kiến trúc

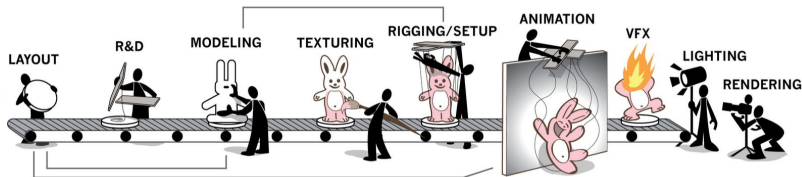
- Mô phỏng công trình một cách chính xác
- Trực quan hóa ý tưởng, hỗ trợ giao tiếp với khách hàng và đối tác

🎮 Điện ảnh & Trò chơi

- Tăng tính chân thực và hấp dẫn cho nội dung
- Cải thiện trải nghiệm người dùng và tối ưu hóa sản phẩm



Computer Graphics Pipeline



Hình 1: Quy trình thiết kế mô hình 3D thủ công¹

¹Nguồn: *3D Animation Essentials* của Andy Beane.

Tự động hóa quy trình 3D

⚠ Nguyên nhân

- Modeling & Texturing tốn nhiều thời gian và công sức

📈 Nhu cầu

- Tăng hiệu suất thiết kế 3D
- Rút ngắn thời gian sản xuất nội dung số



🎯 Mục tiêu

- Hệ thống sinh mô hình 3D từ đầu vào văn bản
- Giảm thời gian cho quá trình Modeling & Texturing

Table of Contents

1 Đặt vấn đề

2 Các hướng giải quyết

- Sinh vật thể 3D: Phương pháp tối ưu hóa
- Sinh vật thể 3D: Phương pháp sinh trực tiếp

3 Phương pháp

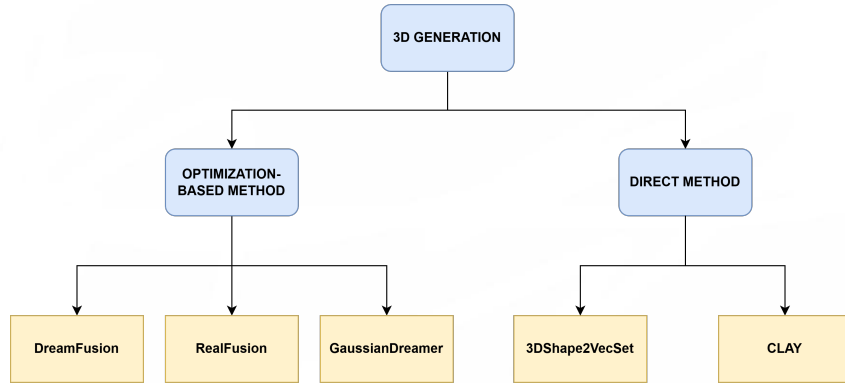
4 Thực nghiệm

- Tập dữ liệu và tiền xử lý
- Các thí nghiệm

5 Đánh giá

6 Kết luận

Tổng quan các phương pháp sinh mô hình 3D



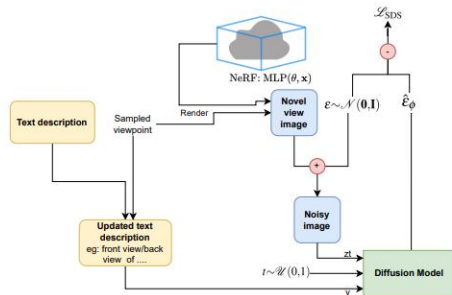
Hình 2: Phân loại các hướng tiếp cận trong bài toán sinh mô hình 3D, tham khảo từ "Advances in 3D Generation: A Survey"[3]

Sinh vật thể 3D: Phương pháp tối ưu hóa

⚙️ Quy trình

- ➔ Khởi tạo mô hình ban đầu bằng NeRF [6] hoặc 3DGS [2].
- ➔ Sử dụng mô hình đã huấn luyện để hướng dẫn việc tối ưu (distillation).
- ➔ Cập nhật tham số sao cho ảnh render khớp với ảnh đầu vào hoặc mô tả yêu cầu.

Đại diện: DreamFusion [9]



Hình 3: Mô hình sinh dữ liệu 3D từ văn bản DreamFusion

Ưu điểm

- ✓ Tận dụng mô hình đã huấn luyện để dẫn hướng tối ưu hoá

Nhược điểm

- ❗ Mỗi đầu vào mới cần huấn luyện lại từ đầu
- ❗ Gây tốn thời gian và tài nguyên tính toán

Sinh vật thể 3D: Phương pháp sinh trực tiếp

Quy trình

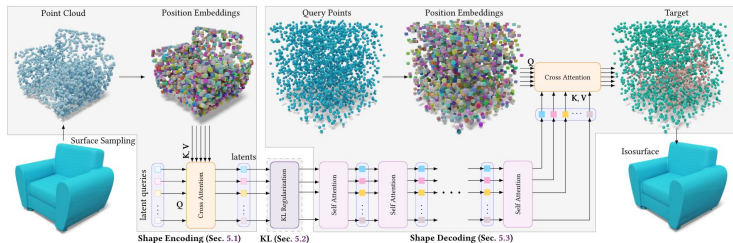
- ➔ VAE [12] học latent space từ dữ liệu 3D
- ➔ Sinh mẫu latent mới sử dụng Diffusion [1] hoặc Flow-based [4]
- ➔ VAE Decoder chuyển latent thành mô hình 3D hoàn chỉnh

3DShape2VecSet [15]

🔧 **Mô tả:** Sinh và hoàn thiện point clouds & meshes từ văn bản, hình ảnh.

💡 **Ý tưởng chính:**

- VAE: Mã hóa và học không gian ẩn của dữ liệu
- Diffusion: Sinh mẫu trên không gian ẩn
- Nhược điểm: Chưa hỗ trợ texture cho mesh sinh ra

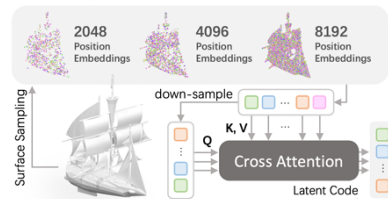


✎ **Mô tả:** Biến văn bản, hình ảnh và các điều kiện khác (multi-view, voxels, point clouds...) thành geometry và texture PBR chất lượng cao

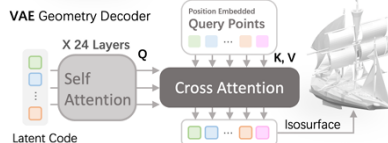
💡 **Ý tưởng chính:**

- Multi-resolution VAE học không gian ẩn (geometry) từ các mô hình 3D
- Diffusion Transformer: sinh mẫu geometry mới từ không gian ẩn
- Multi-view diffusion tạo texture PBR 2K (diffuse, roughness, metallic)

VAE Geometry Encoder



VAE Geometry Decoder



Latent Diffusion Model



Ưu điểm

- ✓ Cho phép tạo mô hình 3D nhanh chóng
- ✓ Tính tổng quát cao, dễ mở rộng với đầu vào như văn bản và hình ảnh

Nhược điểm

- ❗ Xử lý dữ liệu 3D và huấn luyện tốn nhiều tài nguyên tính toán
- ❗ Phụ thuộc vào tập dữ liệu 3D lớn, chất lượng cao

Table of Contents

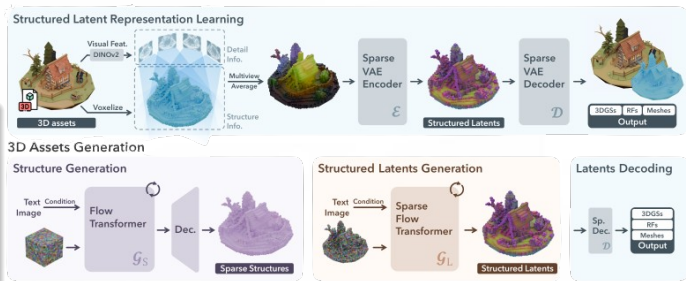
- 1 Đặt vấn đề
- 2 Các hướng giải quyết
 - Sinh vật thể 3D: Phương pháp tối ưu hóa
 - Sinh vật thể 3D: Phương pháp sinh trực tiếp
- 3 Phương pháp
- 4 Thực nghiệm
 - Tập dữ liệu và tiền xử lý
 - Các thí nghiệm
- 5 Đánh giá
- 6 Kết luận

Baseline: TRELLIS [13]

TRELLIS sinh mô hình 3D từ văn bản hoặc hình ảnh dựa trên biểu diễn SLAT (Structured Latent).

💡 Ý tưởng chính

- VAE: Học latent space của các mô hình 3D
- Rectified Flow: Sinh mẫu mới trong latent space



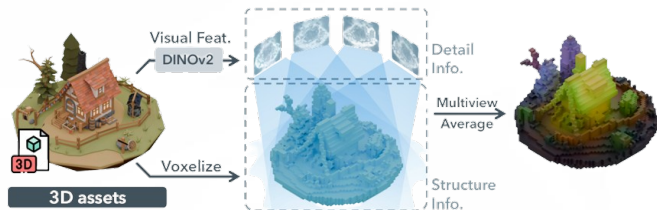
☰ Quy trình

 **Voxelization:** Chuyển mô hình 3D thành tập voxel $\{p_i\}_{i=1}^L$

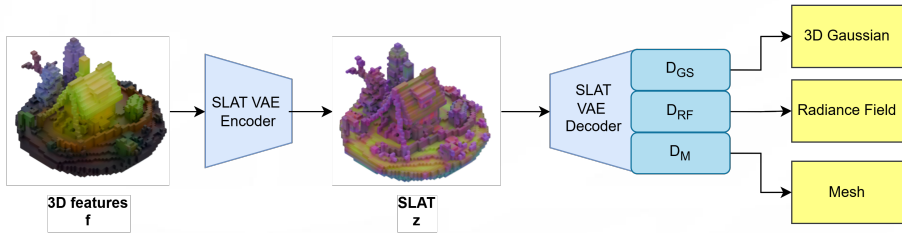
 Trích đặc trưng:

- Render từ nhiều góc nhìn
- Dùng DINOv2 để lấy feature map
- Chiều từng p_i lên ảnh và trung bình hóa để nhận f_i

→ **Output:** Bộ đặc trưng 3D $\{(f_i, p_i)\}_{i=1}^L$

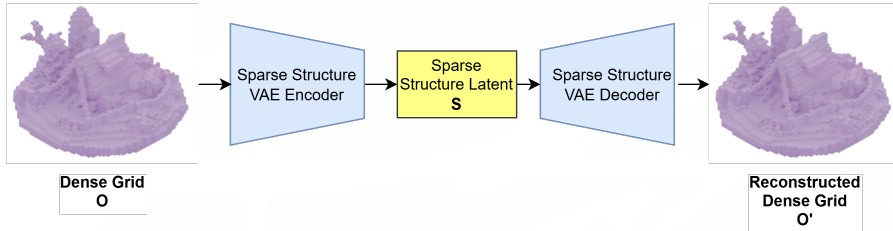


Học biểu diễn SLAT



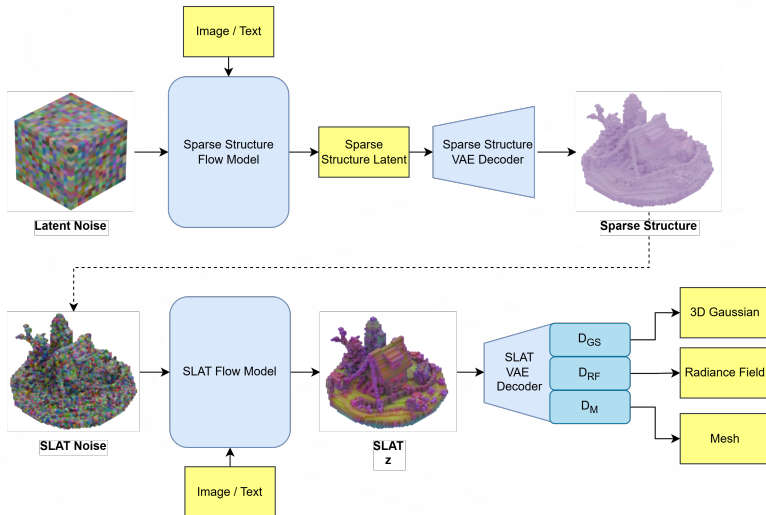
$$\{(f_i, p_i)\}_{i=1}^L \rightarrow \{(z_i, p_i)\}_{i=1}^L \rightarrow \{(\hat{f}_i, p_i)\}_{i=1}^L \rightarrow \begin{cases} \text{3D Gaussian} \\ \text{Radiance Field} \\ \text{Mesh} \end{cases}$$

Học biểu diễn ẩn cấu trúc thưa



$$\{p_i\}_{i=1}^L \rightarrow O \in \{0, 1\}^{N \times N \times N} \rightarrow S \in \mathbb{R}^{C_S \times D \times D \times D}$$

Sinh mẫu trong không gian ẩn



1. Huấn luyện

- Lấy x_0 từ dữ liệu 3D, nhiễu $\epsilon \sim \mathcal{N}(0, I)$
- Chọn $t \sim U(0, 1)$, tạo giá trị trung gian:

$$\hat{x}_t = (1 - t) x_0 + t \epsilon$$

- CFM loss:

$$\mathcal{L}_{\text{CFM}} = E_{x_0, \epsilon, t} \|v_\theta(\hat{x}_t, t, c) - (\epsilon - x_0)\|^2$$

2. Dự đoán

- Classifier-Free Guidance:

$$\hat{v}_\theta(x_t, t, c) = (s + 1) v_\theta(x_t, t, c) - s v_\theta(x_t, t)$$

- Phương pháp Euler:

$$x_{t-\Delta t} \approx x_t - \Delta t \hat{v}_\theta(x_t, t, c)$$

Lặp N bước nhỏ $\rightarrow$ thu mẫu cuối

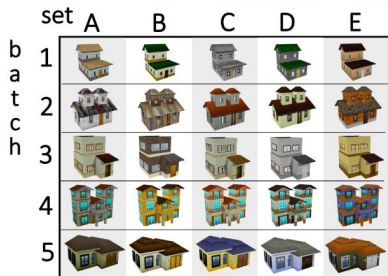
Table of Contents

- 1 Đặt vấn đề
- 2 Các hướng giải quyết
 - Sinh vật thể 3D: Phương pháp tối ưu hóa
 - Sinh vật thể 3D: Phương pháp sinh trực tiếp
- 3 Phương pháp
- 4 **Thực nghiệm**
 - Tập dữ liệu và tiền xử lý
 - Các thí nghiệm
- 5 Đánh giá
- 6 Kết luận

Tập dữ liệu - House3K

Bộ dữ liệu House3K [8]

- Gồm 3000 mô hình nhà với các kết cấu, hình dạng khác nhau.
- [Download](#)



Hình 4: Tập dữ liệu House3K

Raw (FBX)

```
Batch_1
├── Set_A
│   ├── Bat1_SetA_HOUSE1.fbx
│   ├── Bat1_SetA_HOUSE2.fbx
│   ├── ...
│   └── Bat1_SetA_HOUSE50.fbx
├── Set_B
│   ├── ...
├── Set_E
└── Batch_2
    ├── ...
    └── Batch_12
```

Sinh cặp dataset 3D - caption

Quy trình

- ❶ Kết xuất hình ảnh vật 3D từ 4 góc: trước, sau, trái, phải.
- ❷ Sử dụng VLM (Vision language model): gemini-2.0-flash
- ❸ Sinh mô tả từ chi tiết nhất đến ngắn gọn nhất cho từng mô hình 3D.

Kết quả

['Single-story red building with gable roof, windows on all sides, and door.']

'Red one-story house, pitched roof, windows on every side, front door visible.'

'One-story red building, gable roof, windows all around, front door.

...

['Red house: roof, window, door.']



(a) Góc trước



(b) Góc sau



(c) Góc trái



(d) Góc phải

Hình 5: Ví dụ hình ảnh được kết xuất từ bốn góc

Sinh cặp dataset 3D - caption



Carefully summarize in ONE caption aiming for **no more than 40 words** based on the following detailed description. Ensure it's concise and captures the essential information without any extra commentary or unnecessary details. Please avoid hallucination. Provide the caption in a simple, plain text format with bullets, numbering, extra text or special formatting. Detailed description: {detailed_description}



A single-story rectangular building with a pitched gable roof, red-brown exterior walls, and symmetrically placed windows on all sides. The front view includes a door and two windows, while the other sides show consistent window arrangements and structural simplicity.



Now produce exactly TEN versions of that caption, ONE per line, each more brief than the last. DO NOT include any introduction or explanation.

Only output exactly TEN captions, ONE per line. The first should be about 12 words (detailed). The last should be 5 words (super-brief), and it must still capture ****every key object**** from the original.

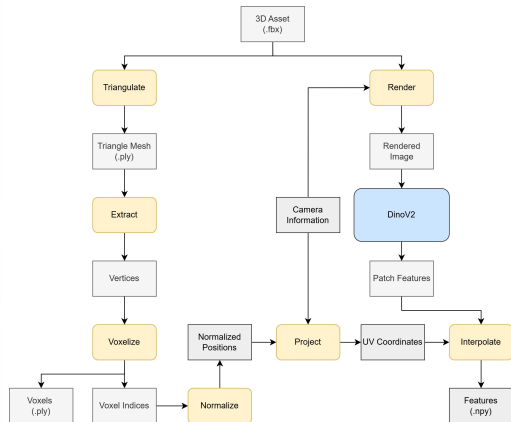


- 1 Single-story red building with gable roof, windows on all sides, and door.
- 2 Red one-story house, pitched roof, windows on every side, front door visible.
- 3 One-story red building, gable roof, windows all around, front door.
- 4 Red building with gable roof, side windows, and a front door.
- 5 Gable-roofed red house with windows and a door.
- 6 Red building, pitched roof, windows, and door.
- 7 One-story red house with windows and door.
- 8 Red house, gable roof, windows, door.
- 9 Red house with roof, windows, door.
- 10 Red house: roof, windows, door.

Trích xuất đặc trưng 3D và thông tin cấu trúc vật thể

Quy trình

- ❶ Kết xuất mô hình 3D ở nhiều góc khác nhau (150 góc nhìn).
- ❷ Voxel hóa mô hình 3D.
- ❸ Sử dụng mô hình DinoV2 [7] để trích xuất đặc trưng cho mỗi góc nhìn và lấy trung bình trên từng voxel.



Hình 7: Quá trình trích xuất đặc trưng (bước 1-3)

Trích xuất đặc trưng 3D và thông tin cấu trúc vật thể

Quy trình

- ④ Mã hóa kiến trúc trúc thô 3D
 - Input: Lưới voxel từ bước 2
 - Mô hình: Sparse Structure Encoder
 - Output: ss feature
- ⑤ Mã hóa thông tin đặc trưng chi tiết
 - Input: Đặc trưng được trích xuất từ bước 3
 - Mô hình: Structured Latent Encoder
 - Output: slat feature

Kết quả

```
House3K
├── features
├── latents
├── raw
├── renders
├── ss_latents
├── voxel
└── metadata.csv
```

Thiết lập thực nghiệm

Chia tập dữ liệu

Tập dữ liệu được xáo trộn ngẫu nhiên sau đó chia theo tỉ lệ:

- 70% dữ liệu train.
- 20% dữ liệu validate.
- 10% dữ liệu test.

Cấu hình phần cứng

- 1 GPU A100 40GB
- 12 CPU 85GB

Các kỹ thuật khác

- Early stopping

Siêu tham số huấn luyện

- learning_rate: $1e-4$
- optimizer: AdamW
- weight_decay = $1e-5$

Các thí nghiệm

- ❶ **Thí nghiệm 1:** Finetune mô hình trên tập dữ liệu mới
- ❷ **Thí nghiệm 2:** Thay thế mô hình sinh
- ❸ **Thí nghiệm 3:** Tinh giản kích thước mô hình bằng kỹ thuật distillation

Thí nghiệm 1: Finetune mô hình trên tập dữ liệu mới

Các kỹ thuật

- 1 Fine-tuning một phần mạng với L2-SP regularization
- 2 Fine-tuning với kỹ thuật LoRA

Mục tiêu

- Giảm chi phí huấn luyện
- Bảo toàn tri thức chung
- Thích nghi nhanh với dữ liệu mới

Tổng quan quá trình fine-tuning TRELIS

- TRELIS gồm 3 giai đoạn:
 - Học biểu diễn SLAT (Structured Latent)
 - Học latent space của cấu trúc thưa (Sparse Structure)
 - Sinh dữ liệu trong hai latent space
- Khi áp dụng cho tập dữ liệu mới:
 - Giữ nguyên giai đoạn 1 và 2 (đóng băng trọng số)
 - Fine-tune giai đoạn sinh dữ liệu

Flow Model - Quá trình huấn luyện

Iterate x_0

- Sample ϵ (noise), t (timestep)

- $x_t = (1 - t)x_0 + t\epsilon$

- $\hat{v} = v_\theta(x_t, t)$

$$v = \epsilon - x_t$$

- $\mathcal{L}_{CFM}(\theta) = \mathbb{E}_{t, x_0, \epsilon} \|v_\theta(x, t) - v\|_2^2$

$$\Leftrightarrow \mathcal{L}_{CFM}(\theta) = \mathbb{E}_{t, x_0, \epsilon} \|v_\theta(x, t) - (\epsilon - x_0)\|_2^2$$

L2-SP Regularization

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{MSE}} + \lambda \sum_i (w_i - w_i^{(0)})^2$$

Trong đó:

- $\mathcal{L}_{\text{MSE}} = \mathcal{L}_{CFM}$: sai số dự đoán
- w_i : trọng số hiện tại
- $w_i^{(0)}$: trọng số ban đầu
- λ : hệ số điều chỉnh

Cấu hình cho fine-tuning: LoRA và L2-SP

LoRA Hai cấu hình cho LoRA được thử nghiệm

1 Cấu hình 1

```
LORA_CONFIG1 = LoraConfig(
    r=4,
    lora_alpha=8.0,
    target_modules=['to_qkv', 'to_q', 'to_kv'],
    lora_dropout=0.05,
    bias="none"
)
```

2 Cấu hình 2

```
LORA_CONFIG2 = LoraConfig(
    r=8,
    lora_alpha=16.0,
    target_modules=['to_qkv', 'to_q', 'to_kv'],
    lora_dropout=0.05,
    bias="none"
)
```

Finetune với L2-SP

1 Sparse Structure Flow Model

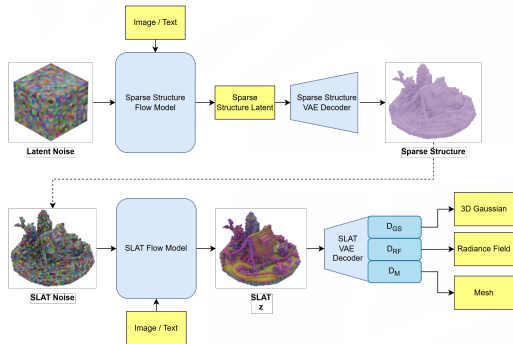
```
freeze_cfg = {
    't_embedder': True,
    'input_layer': False,
    'blocks': 0.3,
    'out_layer': False,
    'adaLN_modulation': False,
}
```

2 Structured Latent Flow Model

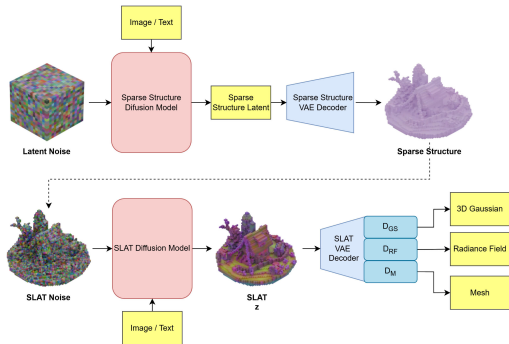
```
freeze_cfg = {
    't_embedder': True,
    'pos_embedder': True,
    'input_layer': False,
    'input_blocks': 0.3,
    'blocks': 0.3,
    'out_blocks': 0.3,
    'out_layer': False,
    'adaLN_modulation': False,
}
```

Thí nghiệm 2: Thay thế mô hình sinh

Huấn luyện: Cùng số bước - 1000 timesteps



(a) Mô hình sinh Rectified Flow



(b) Mô hình sinh Diffusion

Thí nghiệm 3: Knowledge Distillation

Mục tiêu

- Student bắt chước Teacher thông qua kết quả
- Giảm độ phức tạp tính toán khi thực hiện inference

Lợi ích

- Mô hình nhẹ, inference nhanh
- Giữ lại biểu diễn giàu tri thức từ Teacher

Hàm mất mát

Nội suy hai thành phần MSE:

$$\mathcal{L}_{\text{total}} = \alpha \|z_s - z_t\|^2 + (1 - \alpha) \|z_s - z_{gt}\|^2$$

- z_s, z_t, z_{gt} : outputs của student, teacher và ground truth
- α điều chỉnh tầm quan trọng giữa hai thành phần

Table of Contents

- 1 Đặt vấn đề
- 2 Các hướng giải quyết
 - Sinh vật thể 3D: Phương pháp tối ưu hóa
 - Sinh vật thể 3D: Phương pháp sinh trực tiếp
- 3 Phương pháp
- 4 Thực nghiệm
 - Tập dữ liệu và tiền xử lý
 - Các thí nghiệm
- 5 Đánh giá**
- 6 Kết luận

Phương pháp đánh giá

① Đánh giá các mô hình fine-tuning

- So sánh hai cấu hình fine-tuning (LoRA + L2-SP) với mô hình gốc.
- So sánh tác động của mô hình trước và sau khi fine-tuning.

② Đánh giá ảnh hưởng của mô hình sinh Diffusion so với Rectified Flow

③ Đánh giá mô hình distillation



So sánh với các hướng khác:

- **GaussianDreamer [14]**: Một phương pháp dựa trên tối ưu hóa 3D Gaussian Splatting (3DGS) thuộc conference CVPR 2024, đại diện tiêu biểu cho hướng tiếp cận tối ưu hóa (Optimization-based).
- **Stable Diffusion [10] + TRELIS Image-to-3D**: Hướng tiếp cận kết hợp mô hình tạo ảnh với mô hình image-to-3D, tận dụng sức mạnh của các mô hình đã huấn luyện tốt.

Đánh giá - Các mô hình fine-tuning

Finetune	Early Stop	CLIP $\uparrow$	FD _{Incep} $\downarrow$	KD _{Incep} $\downarrow$	FD _{DINO} $\downarrow$	KD _{DINO} $\downarrow$
✓	✗	0.2999	198.54	0.8687	130.48	1.9758
✓	Pat. = 5	0.2939	339.39	0.0593	197.44	11.34
✗	-	0.3047	345.66	1.2782	254.05	1.4435

Bảng 1: Finetune với hàm mất mát gốc ($\mathcal{L}_{\text{MSE}}$)

Finetune	Early Stop	CLIP $\uparrow$	FD _{Incep} $\downarrow$	KD _{Incep} $\downarrow$	FD _{DINO} $\downarrow$	KD _{DINO} $\downarrow$
✓	✗	0.2911	380.589	0.5713	234.35	0.0030
✓	Pat. = 10	0.2883	433.84	0.3258	236.23	1.7890
✗	-	0.3047	345.66	1.2782	254.05	1.4435

Bảng 2: Finetune kết hợp với hàm mất mát L2SP

Finetune	Early Stop	LoRA Config	LR	CLIP $\uparrow$	FD _{Incep} $\downarrow$	KD _{Incep} $\downarrow$	FD _{DINO} $\downarrow$	KD _{DINO} $\downarrow$
✓	✗	LORA_CONFIG1	1e-4	0.3056	246.03	2.5777	196.86	1.9200
✓	Pat. = 5	LORA_CONFIG1	1e-4	0.3053	<u>303.55</u>	1.2312	216.48	1.5325
✓	Pat. = 10	LORA_CONFIG2	1e-4	0.3012	313.48	1.1835	<u>213.64</u>	1.5018
✓	Pat. = 10	LORA_CONFIG2	1e-5	0.3056	352.15	1.2530	251.41	1.4385
✗	-	-	-	0.3047	345.66	1.2782	254.05	<u>1.4435</u>

Bảng 3: Finetune với LoRA

Đánh giá - Thay đổi mô hình sinh

G_{S0}	G_L	CLIP $\uparrow$	FD _{Incep} $\downarrow$	KD _{Incep} $\downarrow$	FD _{DINO} $\downarrow$	KD _{DINO} $\downarrow$
Rectified Flow	Rectified Flow	0.3047	345.66	1.2782	254.05	1.4435
Diffusion	Diffusion	0.2104	1736.49	3.7442	539.85	0.5428

Bảng 4: So sánh các mô hình sinh

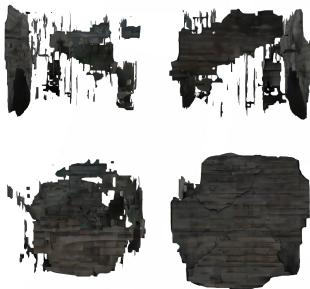
Đánh giá - Thay đổi mô hình sinh

Prompt: "Stylized modern house: dark brick, wood, concrete, flat roof, double doors."

Diffusion-based

Sampling: 700 steps

C



Flow-based

Sampling: 25 steps



Đánh giá mô hình distillation

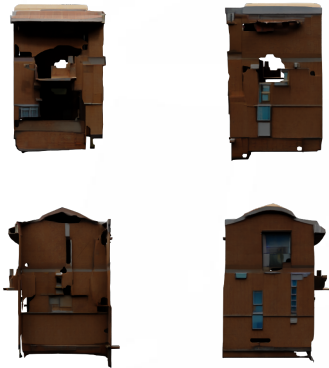
Mô hình	num. params G_S	num. params G_L	Avg. time	CLIP $\uparrow$	$FD_{Incep} \downarrow$	$KD_{Incep} \downarrow$	$FD_{DINO} \downarrow$	$KD_{DINO} \downarrow$
Teacher	156,726,536	185,364,616	8.9933 secs	0.3047	345.66	1.2782	245.05	1.4435
Student	185,364,616	25,832,968	2.7312 secs	0.2505	843.68	1.3040	373.65	0.0011

Bảng 5: Đánh giá mô hình học với distillation

Đánh giá mô hình distillation

Prompt: "Stylized building: beige walls, red roof, turquoise windows, brick accents."

Student



Teacher



Đánh giá mô hình distillation

Prompt: "Stylized 3D house: peach stucco, stone base, gray roof, barred windows, carport."

Student



Teacher



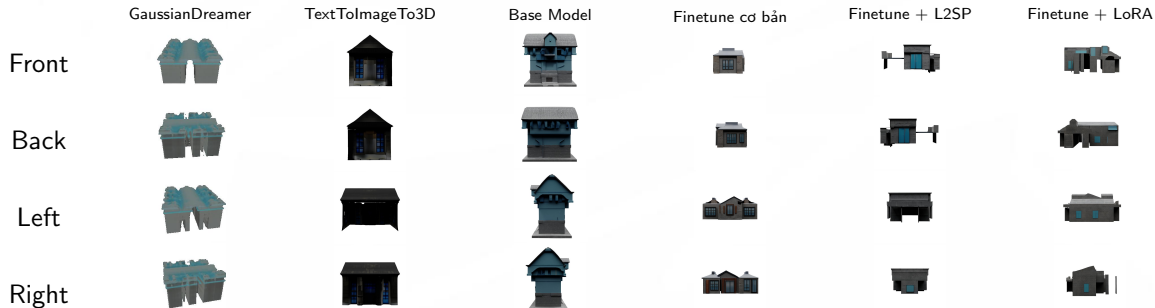
So sánh với phương pháp khác

Mô hình	CLIP $\uparrow$	FD _{Incep} $\downarrow$	KD _{Incep} $\downarrow$	FD _{DINO} $\downarrow$	KD _{DINO} $\downarrow$
TRELLIS _{text} [13]	0.3047	0.34566	1.2782	254.05	1.4435
StableDiff. [11] + TRELLIS _{image} [13]	0.2568	498.33	1.2593	319.26	1.3966
GaussianDreamer [14]	0.2582	1410.72	1.5405	571.34	1.5039

Bảng 6: So sánh với các phương pháp khác

Một số kết quả mô phỏng

Prompt: "Stylized 3D building with orange roof, brick walls, blue door, white pillars."



Một số kết quả mô phỏng

Prompt: "Minimalist 3D house: geometric design, grey/brown tones, rectangular windows, door, tower."

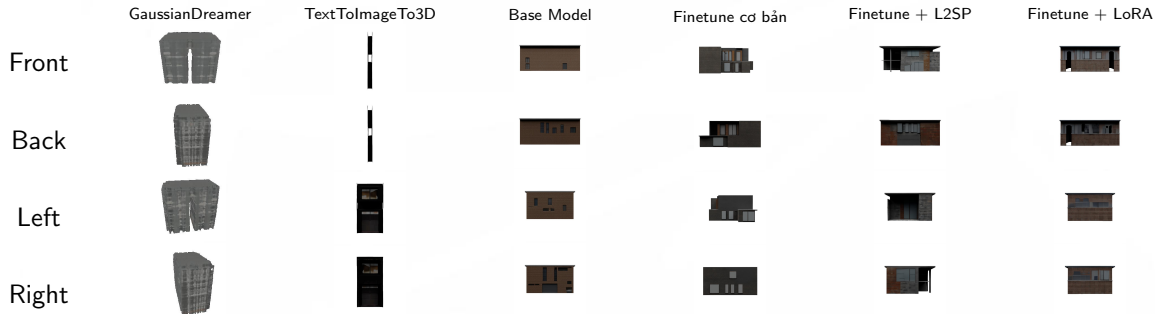
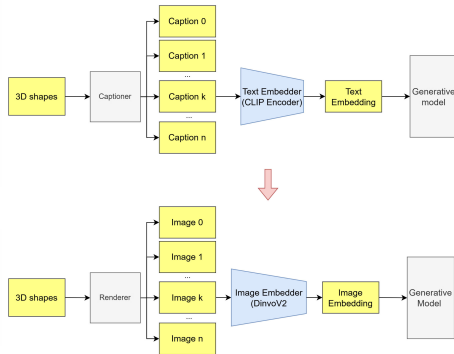


Table of Contents

- 1 Đặt vấn đề
- 2 Các hướng giải quyết
 - Sinh vật thể 3D: Phương pháp tối ưu hóa
 - Sinh vật thể 3D: Phương pháp sinh trực tiếp
- 3 Phương pháp
- 4 Thực nghiệm
 - Tập dữ liệu và tiền xử lý
 - Các thí nghiệm
- 5 Đánh giá
- 6 Kết luận



Hình 15: Hướng triển khai cho dữ liệu đầu vào dạng hình ảnh

Tổng kết Đồ án

Những điểm đã đạt được:

- 1 Đã áp dụng thành công kỹ thuật fine-tune (LoRA, L2-SP) để cải thiện hiệu suất sinh mô hình 3D từ văn bản:
 - **LoRA**: Cải thiện rõ rệt độ khớp giữa mô tả văn bản và mô hình 3D, giúp giảm sai lệch thị giác (FID, KID) $\Rightarrow$ Phù hợp cho mục tiêu tối ưu chất lượng sinh ảnh.
 - **L2-SP**: Duy trì độ ổn định của mô hình và giảm nguy cơ overfitting. Tuy nhiên, mức cải thiện không cao bằng LoRA.
- 2 Đã triển khai thành công kỹ thuật **Knowledge Distillation**, giúp:
 - Giảm đáng kể thời gian inference.
 - Mặc dù hiệu suất có giảm so với mô hình gốc, nhưng vẫn đạt chất lượng chấp nhận được $\Rightarrow$ Có tiềm năng ứng dụng trong các hệ thống thời gian thực và thiết bị tài nguyên thấp.
- 3 Đã xây dựng pipeline để tổng hợp cặp dữ liệu 3D-mô tả cho tập dữ liệu mới (Houses3K), có khả năng mở rộng cho các tập dữ liệu đặc thù khác.

Những điểm hạn chế:

- 1 **Diffusion models** không cho kết quả tốt trên miền không gian ẩn của bài toán này:
 - Số bước sinh nhiều, nhưng chất lượng đầu ra không đạt kỳ vọng.
 - Có thể do chưa phù hợp với đặc thù dữ liệu.

Hướng phát triển trong tương lai:

- Nâng cấp mô hình sinh để xử lý các mô tả văn bản phức tạp và phong phú hơn.
- Mở rộng với dữ liệu đầu vào dạng hình ảnh.
- Tăng cường chi tiết hình học và tích hợp khả năng tương tác người dùng (điều khiển kết quả sinh theo thời gian thực).

Xin chân thành cảm ơn!

- [1] Jonathan Ho **and others**. “Denoising diffusion probabilistic models”. in *NeurIPS*: (2020).
- [2] Bernhard Kerbl **and others**. “3D Gaussian Splatting for Real-Time Radiance Field Rendering”. in *ACM Transactions on Graphics (TOG)*: 42.4 (2023), 139:1–139:14. DOI: 10.1145/3592433.
- [3] Xiaoyu Li **and others**. *Advances in 3D Generation: A Survey*. 2024. arXiv: 2401.17807 [cs.CV]. URL: <https://arxiv.org/abs/2401.17807>.
- [4] Yaron Lipman **and others**. *Flow Matching for Generative Modeling*. 2023. arXiv: 2210.02747 [cs.LG]. URL: <https://arxiv.org/abs/2210.02747>.
- [5] Xingchao Liu, Chengyue Gong **and** Qiang Liu. *Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow*. 2022. arXiv: 2209.03003 [cs.LG]. URL: <https://arxiv.org/abs/2209.03003>.

- [6] Ben Mildenhall **and others**. “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis”. *in Proceedings of the European Conference on Computer Vision (ECCV)*: Springer, 2020.
- [7] Maxime Oquab **and others**. *DINOv2: Learning Robust Visual Features without Supervision*. 2024. arXiv: 2304.07193 [cs.CV]. URL: <https://arxiv.org/abs/2304.07193>.
- [8] Daryl Peralta. *Houses3K: A dataset of 3000+ house images and features*. <https://github.com/darylperalta/Houses3K>. Accessed: April 26, 2025. 2022.
- [9] Ben Poole **and others**. “Dreamfusion: Text-to-3D Using 2D Diffusion”. *in Proceedings of the International Conference on Learning Representations (ICLR)*: 2022.
- [10] Robin Rombach **and others**. *High-Resolution Image Synthesis with Latent Diffusion Models*. 2022. arXiv: 2112.10752 [cs.CV]. URL: <https://arxiv.org/abs/2112.10752>.

Tài liệu tham khảo III

- [11] Robin Rombach **and others**. “High-Resolution Image Synthesis with Latent Diffusion Models”. *in IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*: 2022.
- [12] Lilian Weng. “From Autoencoder to Beta-VAE”. *in lilianweng.github.io*: (2018). URL: <https://lilianweng.github.io/posts/2018-08-12-vae/>.
- [13] Jianfeng Xiang **and others**. *Structured 3D Latents for Scalable and Versatile 3D Generation*. 2025. arXiv: 2412.01506 [cs.CV]. URL: <https://arxiv.org/abs/2412.01506>.
- [14] Taoran Yi **and others**. *GaussianDreamer: Fast Generation from Text to 3D Gaussians by Bridging 2D and 3D Diffusion Models*. 2024. arXiv: 2310.08529 [cs.CV]. URL: <https://arxiv.org/abs/2310.08529>.
- [15] Biao Zhang **and others**. *3DShape2VecSet: A 3D Shape Representation for Neural Fields and Generative Diffusion Models*. 2023. arXiv: 2301.11445 [cs.CV]. URL: <https://arxiv.org/abs/2301.11445>.

- [16] Longwen Zhang **and others**. *CLAY: A Controllable Large-scale Generative Model for Creating High-quality 3D Assets*. 2024. arXiv: 2406.13897 [cs.CV]. URL: <https://arxiv.org/abs/2406.13897>.

Table of Contents

7 Appendix

Volume Rendering

Sau khi tính toán được các giá trị mật độ và màu sắc tại những điểm lấy mẫu, ta kết xuất hình ảnh bằng công thức kết xuất thể tích (Volume Rendering)

$$C(r) = \int_{t_n}^{t_f} T(t) \sigma(r(t)) c(r(t), \mathbf{d}) dt, \text{ với}$$

$$T(s) = \exp \left(- \int_{t_n}^s \sigma(r(s)) ds \right)$$

Xấp xỉ bằng tổng Riemann:

$$\hat{C}(r) = \sum_{i=1}^N T_i (1 - \exp(-\sigma_i \theta_i)) c_i, \text{ với}$$

$$T_i = \exp \left(- \sum_{j=1}^{i-1} \sigma_j \delta_j \right)$$

Variational Autoencoder (VAE)

Mục tiêu: Học phân phối xác suất của dữ liệu bằng cách tối ưu hóa hàm mất mát ELBO.

Công thức ELBO:

$$\mathcal{L} = \mathbb{E}_{q(\mathbf{z}|\mathbf{x})} [\log p(\mathbf{x} | \mathbf{z})] - \text{KL} (q(\mathbf{z} | \mathbf{x}) \parallel p(\mathbf{z}))$$

- $q(\mathbf{z} | \mathbf{x})$: Phân phối posterior của không gian ẩn $\mathbf{z}$.
- $p(\mathbf{x} | \mathbf{z})$: Mô hình likelihood để tái tạo dữ liệu $\mathbf{x}$.
- KL: Divergence Kullback-Leibler giữa phân phối posterior và prior $p(\mathbf{z})$.

Mục đích: Tối thiểu hóa KL divergence và tối đa hóa log-likelihood của dữ liệu.

Quá trình khuếch tán thuận (Forward Process)

Quá trình khuếch tán thuận được mô tả bằng công thức:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

- $\mathbf{x}_t$: Dữ liệu tại thời điểm t .
- $\beta_t \in (0, 1)$: Variance Scheduler tại thời điểm t , kiểm soát mức độ nhiễu trong quá trình khuếch tán.

Quá trình này được lặp lại cho các bước $t = 1, 2, \dots, T$, tạo ra dữ liệu nhiễu từ $\mathbf{x}_0$.

Quá trình khuếch tán thuận (Forward Process)

Mục tiêu: Mô phỏng quá trình khuếch tán bằng cách biến đổi $\mathbf{x}_0$ thành nhiễu trắng.

Tại mỗi bước t , dữ liệu $\mathbf{x}_t$ có thể được viết như sau:

$$\mathbf{x}_t = \sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \epsilon$$

Trong đó:

- $\alpha_t = \prod_{i=1}^t (1 - \beta_i)$, đây là yếu tố kiểm soát sự tích lũy nhiễu.
- $\epsilon \sim \mathcal{N}(0, I)$ là nhiễu Gaussian được thêm vào tại mỗi bước.

Quá trình nghịch (Reverse Process)

Mục tiêu: Học cách loại bỏ nhiễu từ dữ liệu $\mathbf{x}_t$ để tái tạo lại $\mathbf{x}_0$.

Quá trình ngược mô tả việc loại bỏ nhiễu từ dữ liệu đã khuếch tán:

$$p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_{\theta}(\mathbf{x}_t, t), \Sigma_{\theta}(\mathbf{x}_t, t))$$

- $\mu_{\theta}(\mathbf{x}_t, t)$: Dự đoán giá trị dữ liệu tại bước $t - 1$ dựa trên dữ liệu tại bước t .
- $\Sigma_{\theta}(\mathbf{x}_t, t)$: Phương sai của phân phối, mô hình sự không chắc chắn trong quá trình phục hồi.

Quá trình này giúp loại bỏ nhiễu và phục hồi dữ liệu gốc từ nhiễu trắng.

Hàm mất mát trong Diffusion

Mục tiêu: Tối thiểu hóa sự khác biệt giữa nhiễu dự đoán và nhiễu thực tế.

Hàm mất mát được sử dụng trong Diffusion để học dự đoán nhiễu $\epsilon_{\theta}(\mathbf{x}_t, t)$ tại mỗi bước:

$$\mathcal{L} = \mathbb{E}_q [\|\epsilon - \epsilon_{\theta}(\mathbf{x}_t, t)\|^2]$$

- ϵ : Nhiễu thực tế (Gaussian).
- $\epsilon_{\theta}(\mathbf{x}_t, t)$: Nhiễu dự đoán bởi mô hình.

Quá trình tối ưu hóa hàm mất mát này giúp mô hình học cách tái tạo dữ liệu gốc từ nhiễu trắng.

Chamfer Distance

Đo độ giống nhau giữa hai tập điểm bằng cách đo chênh lệch khoảng cách giữa hai tập điểm đó

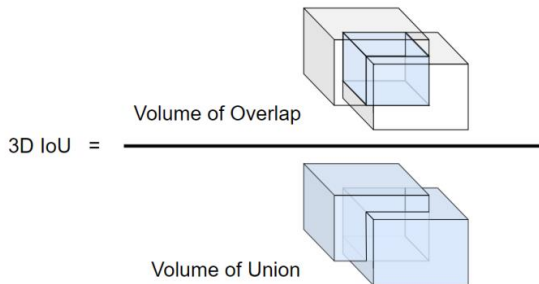
$$CD(R, G) = \frac{1}{|R|} \sum_{r \in R} \min_{g \in G} \|g - r\|^2 + \frac{1}{|G|} \sum_{g \in G} \min_{r \in R} \|r - g\|^2$$

$$\text{F-score} = 2 \frac{\text{precision} * \text{recall}}{\text{precision} + \text{recall}}$$

Thường có dạng F1@x, trong đó x là giá trị ngưỡng khoảng cách, ví dụ F1@0.1, F1@0.2, F1@0.5, ...

Volume IoU

Đo độ chồng thể tích của mô hình dự đoán và mô hình thực tế

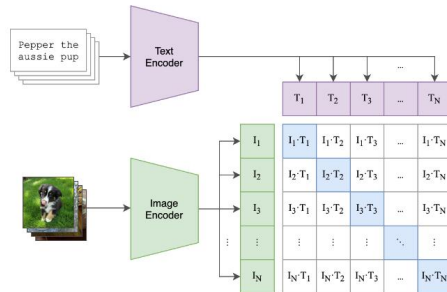


Hình 16: Volume IoU

Nguồn: <https://arxiv.org/pdf/2006.14837>:

CLIP Score

Tận dụng mô hình CLIP đã được huấn luyện, tính độ giống cosine (cosine similarity) giữa điều kiện đầu vào (hình ảnh, văn bản) và hình ảnh kết xuất từ mô hình 3D dự đoán



Hình 17: CLIP model

Nguồn: <https://arxiv.org/pdf/2103.00020>

Minimum Matching Distance

Tính toán các khối 3D được sinh có gần với các khối 3D trong bộ dữ liệu thực tế. Độ đo này không tính cho từng khối 3D, mà nó được tính toán trên toàn bộ tập dữ liệu.

$$MMD(S_g, S_r) = \frac{1}{|S_r|} \sum_{Y \in S_r} \min_{X \in S_g} D(X, Y)$$

- Với mỗi khối 3D được sinh ra, tính khoảng cách Chamfer giữa nó và toàn bộ khối 3D trong bộ dữ liệu thực tế. Sau đó lấy khoảng cách nhỏ nhất tương ứng với khối 3D này.
- MMD được tính bằng giá trị trung bình giữa các giá trị khoảng cách nhỏ nhất trên toàn bộ tập dữ liệu 3D được sinh ra.

Coverage (COV)

Đánh giá bộ dữ liệu sinh ra có bao phủ được độ đa dạng của bộ dữ liệu thực tế không.

$$COV(S_g, S_r) = \frac{|\{ \operatorname{argmin}_{y \in S_r} D(X, Y) | X \in S_g \}|}{|S_r|}$$

Giá trị COV thấp chứng tỏ mô hình sinh chưa bao quát được bộ dữ liệu thực tế.

Kernel Inception Distance (KID)

- Các hình ảnh được kết xuất từ mô hình dự đoán và hình ảnh đầu vào được đi qua mạng Inception (Inception Model). Đây là một mạng được huấn luyện trên bộ dữ liệu ImageNet
- Đầu ra của quá trình này là các véc-tơ đặc trưng.
- Sử dụng một hàm kernel để tính toán độ giống nhau của từng cặp véc-tơ đặc trưng từ mô hình Inception.

$$KID = MMD^2 = \frac{1}{m(m-1)} \sum_{i \neq j} k(x_i, x_j) + \frac{1}{n(n-1)} \sum_{i \neq j} k(y_i, y_j) - \frac{2}{mn} \sum_{i,j} k(x_i, y_j)$$

KID càng thấp thì mô hình càng tốt

Fréchet Inception Distance (FID)

Tương tự KID, tuy nhiên thay vì sử dụng hàm kernel thì FID sẽ khớp các véc-tơ đặc trưng được sinh ra từ mô hình Inception vào hai phân phối chuẩn tương ứng với dữ liệu thực và dữ liệu sinh. FID được tính dựa vào hai phân phối này.

$$FID = ||\mu_r - \mu_g||^2 + Tr(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{\frac{1}{2}})$$

Peak-Signal-to-Noise Ratio (PSNR)

So sánh giữa ảnh điều kiện đầu vào và ảnh kết xuất từ mô hình 3D được sinh ra.

$$PSNR = 10\log_{10}\left(\frac{MAX_I^2}{MSE}\right)$$

Trong đó, MAX_I giá trị lớn nhất có thể có của ô điểm ảnh trong miền tương ứng (ví dụ, đối với ảnh 8-bit, giá trị này là 255).

Structural Similarity Index Measure (SSIM)

Tính toán sự giống nhau giữa ảnh kết xuất nhiều góc nhìn và dữ liệu kết xuất thực tế.

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

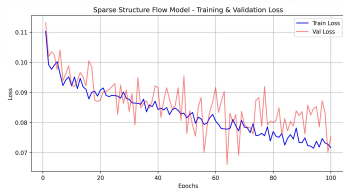
- Độ đo này tập trung vào các thông tin mang tính cấu trúc, gồm độ sáng, độ tương phản và giống nhau về cấu trúc dữ liệu.
- Độ đo này có giá trị nằm trong khoảng -1 đến 1, trong đó 1 là giá trị tối ưu.

Các độ đo KID, FID, PSNR, SSIM cũng có thể được dùng để đánh giá độ tốt của việc tái dựng.

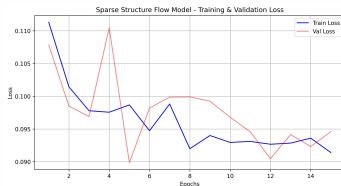
Textual Inversion: Là việc thiết kế một mô tả $e^{(I_0)}$ để cung cấp thông tin cho việc tổng hợp các góc nhìn mới. Việc này được thực hiện bằng các bước như sau:

- Với I_0 là hình ảnh đầu vào, thực hiện gia tăng dữ liệu ảnh này bằng khối augmentation $g(I_0)$. Việc này tạo ra một bộ dữ liệu $\mathcal{D}' = \{g(I_0)\}$
- Tối ưu hàm mất mát của mô hình khuếch tán trên mô tả $e^{(I_0)}$ trong khi đóng băng toàn bộ các tham số của mô hình khuếch tán. Việc này đồng nghĩa với việc đi tìm một vec-tơ văn bản (text embedding) mà có khả năng sinh ra hình ảnh của bộ dữ liệu hình ảnh gia tăng ở bước trên là cao nhất.

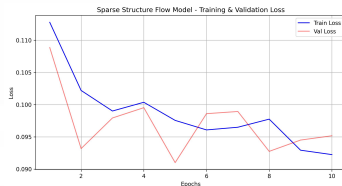
Kết quả huấn luyện: SS Model - Finetune cơ bản



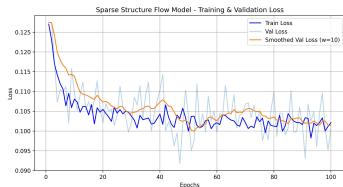
(a) L1, no early stopping



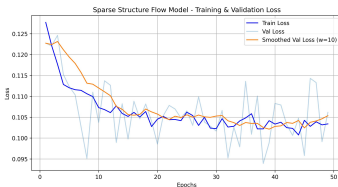
(b) L1, patience=10



(c) L1, patience=5



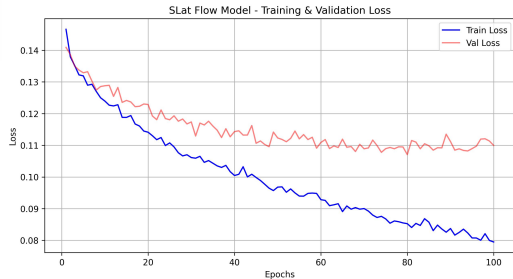
(d) L2SP, no early stopping



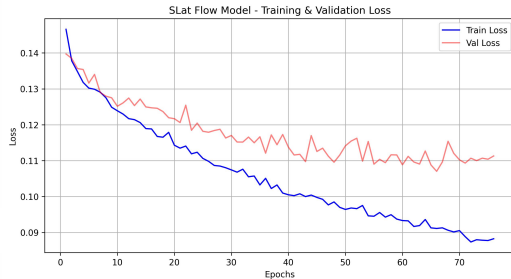
(e) L2SP, patience=10

Hình 18: SS Model - Finetune cơ bản (L1 và L2SP)

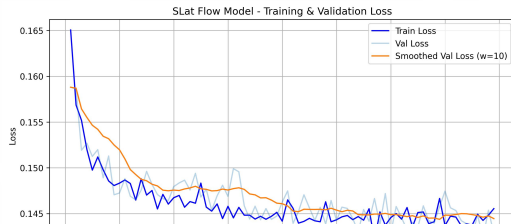
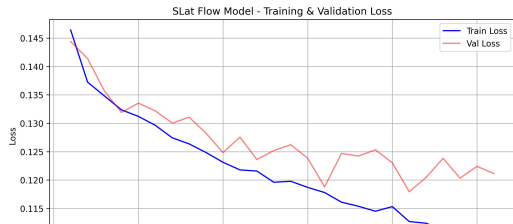
Kết quả huấn luyện: SLat Model - Finetune cơ bản



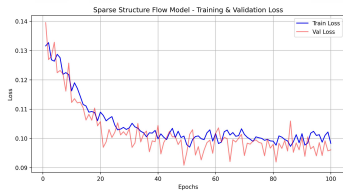
(a) L1, no early stopping



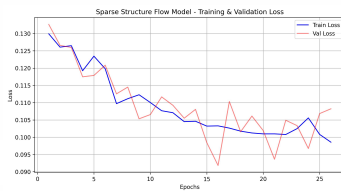
(b) L1, patience=10



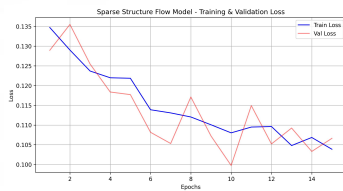
Kết quả huấn luyện: SS Model - LoRA



(a) No early stopping



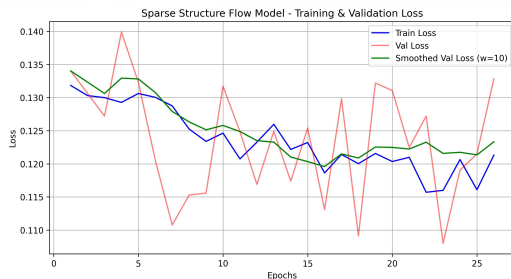
(b) Patience=10



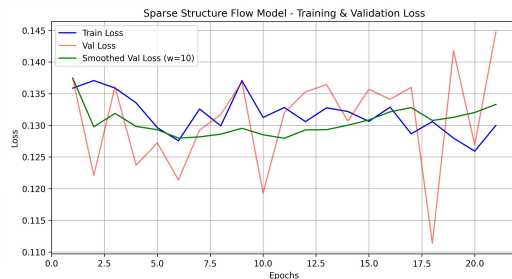
(c) Patience=5

Hình 20: SS Model - Finetune với LoRA

Kết quả huấn luyện: SS Model - LoRA



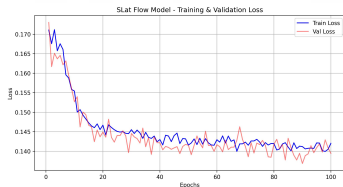
(a) LR=1e-4, patience=10



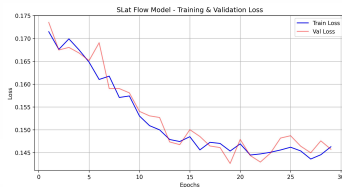
(b) LR=1e-5, patience=10

Hình 21: SS Model - LoRA với Learning Rate

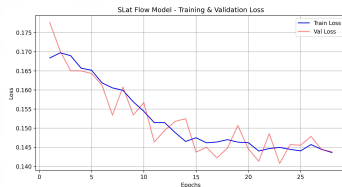
Kết quả huấn luyện: SLat Model - LoRA



(a) No early stopping



(b) Patience=10

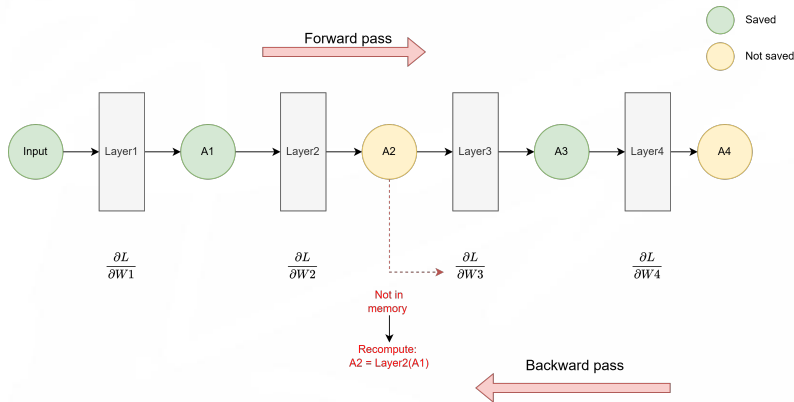


(c) Patience=5

Hình 22: SLat Model - Finetune với LoRA

Gradient Checkpointing

- Thay vì lưu tất cả activations, ta chỉ lưu **checkpoint** vào những layer cụ thể.
- Khi thực hiện backward, tính lại các activations từ checkpoint thay vì lưu hết trong bộ nhớ.



Hình 23: Gradient Checkpointing

Fine-tuning với L2-SP Regularization

Ý tưởng chính

- Chỉ fine-tune một số lớp cuối của decoder
- Dùng regularization để giữ trọng số gần với pretrained

Ưu điểm

- Hạn chế overfitting
- Duy trì kiến thức đã học

Hàm mất mát

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{MSE}} + \lambda \sum_i (w_i - w_i^{(0)})^2$$

Trong đó:

- $\mathcal{L}_{\text{MSE}}$: sai số dự đoán
- w_i : trọng số hiện tại
- $w_i^{(0)}$: trọng số ban đầu
- λ : hệ số điều chỉnh

Fine-tuning với Low-Rank Adaptation (LoRA)

Nguyên lý

- Thêm adapter rank thấp vào các trọng số attention
- Chỉ train adapter, giữ nguyên các trọng số cũ

Ưu điểm

- Số lượng tham số thấp
- Hiệu quả cao trong fine-tuning

Công thức

LoRA

$$\Delta W = BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}$$

Hàm mất mát

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{MSE}}.$$

Áp dụng:

- Vào các trọng số W_q, W_k, W_v của Transformer
- $r \ll \min(d, k)$: đảm bảo rank thấp